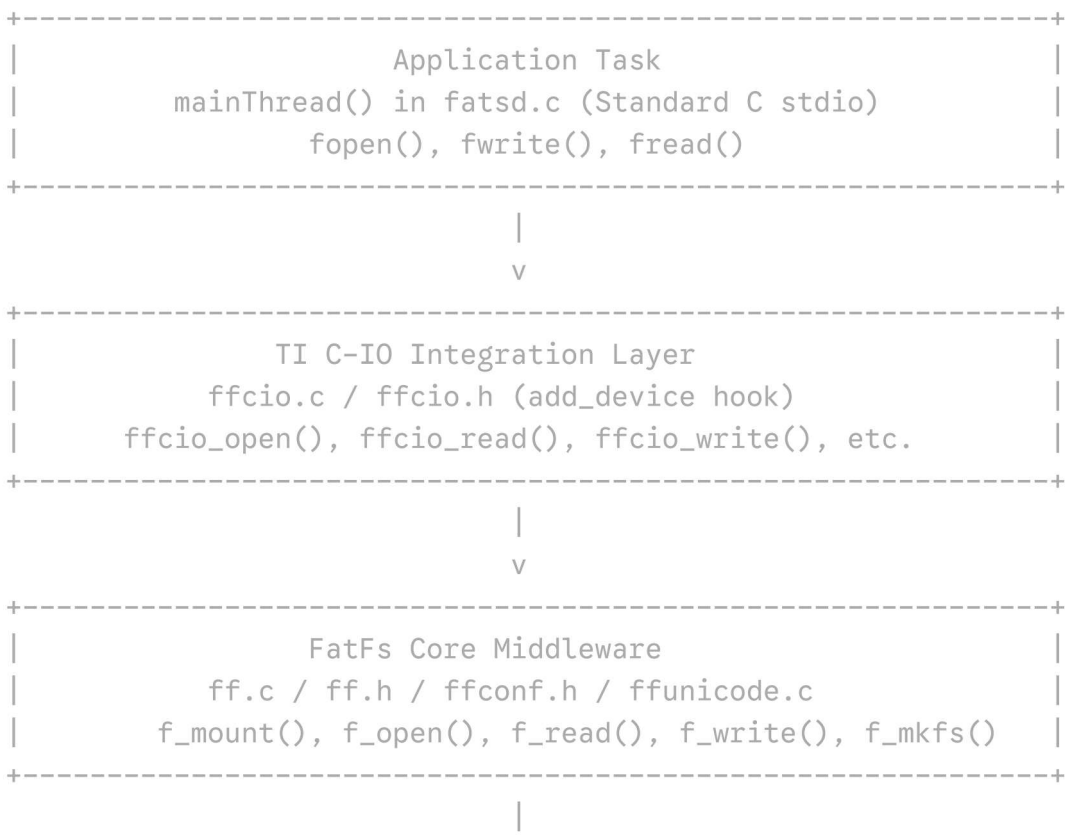
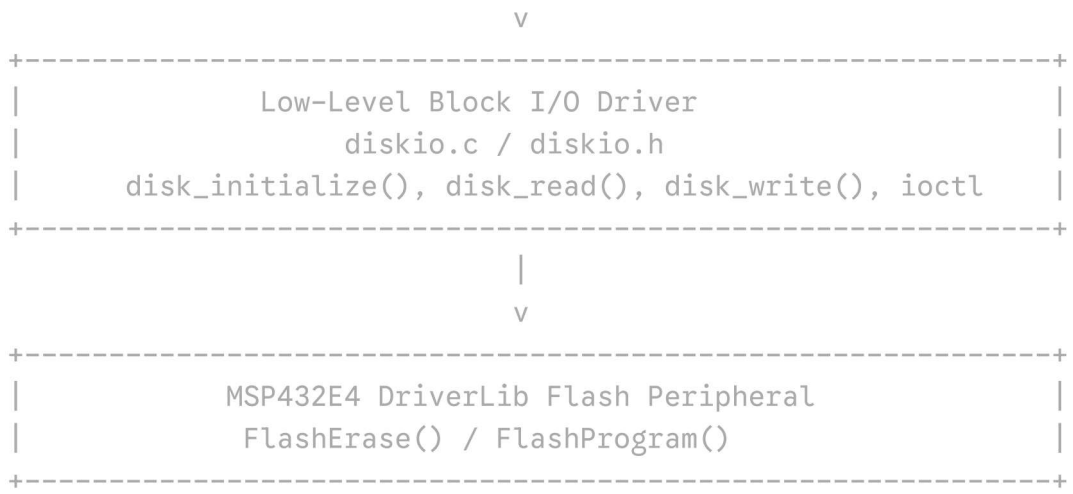
fatsd main_freertos diskio diskio ff ff

explain each file seperately and about functions and complete overall project description in detail

Overall Project Architecture

This project implements an embedded **FAT filesystem in MSP432E4 on-chip flash memory** running on **FreeRTOS**. Instead of using an external SD card or SPI NOR flash, the project dedicates a 256 KB slice of internal microcontroller flash (from address `0x000C0000` to `0x00100000`) as a virtual block device formatted with the FAT filesystem. fatsd.c + 4





Key Architecture Highlights

Storage Emulation: FatFs operates on standard 512-byte logical sectors. However, the MSP432E4 hardware flash sector erase block size is 16 KB (`16384` bytes). The low-level disk I/O driver solves this discrepancy by implementing a **16 KB RAM bounce buffer** and an on-demand **Read-Modify-Write (RMW)** cycle. diskio.c + 3

Standard C-IO Binding: TI RTS `add_device()` maps high-level ANSI C file operations (`fopen()` , `fread()` , `fwrite()` , `fclose()`) to the FatFs APIs via device prefix `fat0:` . fatsd.c + 1

RTOS & Thread-Safety: The FreeRTOS scheduler runs a detached POSIX thread (`pthread`) executing `mainThread` . FatFs is configured with re-entrancy enabled (`FF_FS_REENTRANT = 1`), backed by FreeRTOS binary/counting mutex primitives (`ff_cre_syncobj` , `ff_req_grant` , `ff_rel_grant`). main_freertos.c + 2

Detailed File-by-File & Function Breakdown

1. `main_freertos.c`

This file serves as the system entry point (`main`), hardware board initialization hub, and RTOS thread launcher. main_freertos.c

Functions

`main()` : main_freertos.c

Calls `Board_init()` to initialize clock trees, power domains, and peripheral pins defined in TI-Drivers board configurations. main_freertos.c

Configures POSIX thread attributes using `pthread_attr_init()`, setting the thread stack size to 1024 bytes (`THREADSTACKSIZE`) and execution priority to 1 in detached mode (`PTHREAD_CREATE_DETACHED`).
main_freertos.c

Spawns `mainThread` via `pthread_create()`. main_freertos.c

Starts the FreeRTOS multitasking kernel scheduler via `vTaskStartScheduler()`. main_freertos.c

`vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)` : main_freertos.c

A FreeRTOS diagnostic callback triggered if any task overflows its allocated stack frame. Traps CPU execution in an infinite `while(1)` loop.
main_freertos.c + 1

2. `fatsd.c`

The central application logic file demonstrating volume mounting, automatic partition formatting, file writing, copying, and standard output streaming. fatsd.c

Functions

`mainThread(void *arg0)` : fatsd.c

Initializes board GPIOs (`GPIO_init()`) and UART Display (`Display_init()`, `Display_open()`). fatsd.c

Calls `add_device()` to register the TI RTS C-IO device prefix `"fat"`, binding standard C runtime file functions (`fopen`, `fread`, etc.) to the `ffcio_*` wrappers. fatsd.c

Sets the real-time clock baseline via `clock_settime(CLOCK_REALTIME, &ts)` and toggles `CONFIG_GPIO_LED_0`. fatsd.c

Mounts logical drive `0:` via `f_mount(&g_fatFs, "0:", 1)`. If no filesystem exists (`FR_NO_FILESYSTEM`), it initializes and formats the 256 KB partition using `f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf))` and remounts. fatsd.c + 1

Verifies if `"fat:0:input.txt"` exists; creates and populates it with a predefined string `textarray` if absent. fatsd.c

Performs block-by-block file copying from `input.txt` to `output.txt` using a 2048-byte intermediate buffer (`cpy_buff`). fatsd.c

Re-opens `output.txt` , reads its contents, streams them to the UART console via `Display_printf()` , and unmounts the drive with `f_mount(NULL, "0:", 0)` . fatsd.c

`fatfs_getFatTime()` **and** `get_fattime()` : fatsd.c

Converts standard POSIX epoch seconds (`time(NULL)`) via `localtime()` into the packed 32-bit FAT timestamp bitmask:

Bits: Year offset from 1980 fatsd.c

Bits: Month (1–12) fatsd.c

Bits: Day of month (1–31) fatsd.c

Bits: Hour (0–23) fatsd.c

Bits [10:5]: Minute (0–59) fatsd.c

Bits [4:0]: Seconds divided by 2 (0–29) fatsd.c

FreeRTOS Synchronization Primitives (`#if FF_FS_REENTRANT` **):** fatsd.c

`ff_cre_syncobj(BYTE vol, FF_SYNC_t *sobj)` : Allocates a FreeRTOS mutex using `xSemaphoreCreateMutex()` . fatsd.c

`ff_del_syncobj(FF_SYNC_t sobj)` : Deletes the mutex using `vSemaphoreDelete()` . fatsd.c

`ff_req_grant(FF_SYNC_t sobj)` : Acquires the volume mutex with timeout `pdMS_TO_TICKS(FF_FS_TIMEOUT)` . fatsd.c

`ff_rel_grant(FF_SYNC_t sobj)` : Releases the mutex using `xSemaphoreGive()` . fatsd.c

3. `diskio.c`

The low-level hardware abstraction layer (HAL) connecting FatFs sector-based requests to the internal MSP432E4 NOR flash memory. diskio.c

Configuration Constants

`FLASH_FS_BASE_ADDR (0x000C0000UL)` : Flash physical start address (allocates the upper 256 KB of 1024 KB flash). diskio.c

`FLASH_FS_TOTAL_SIZE (256 KB)` : Total filesystem footprint. diskio.c

`FLASH_SECTOR_SIZE (512 bytes)` : Logical sector size reported to FatFs. diskio.c

`FLASH_ERASE_BLOCK_SIZE (16384 bytes / 16 KB)` : Smallest physical erasable flash unit. diskio.c

`FLASH_SECTOR_COUNT (512 sectors)` : Total addressable sectors ($256\ \text{KB}/512\ \text{B}$). diskio.c

Functions

`disk_initialize(BYTE pdrv)` : diskio.c

Validates drive number (`pdrv == 0`) and returns ready status (`0`). diskio.c

`disk_status(BYTE pdrv)` : diskio.c

Checks drive initialization state; returns `STA_NOINIT` for unhandled drives. diskio.c

`disk_read(BYTE pdrv, BYTE *buff, DWORD sector, UINT count)` : diskio.c

Reads contiguous sectors directly from memory-mapped flash into `buff` using `memcpy()` starting at address `FLASH_FS_BASE_ADDR + (sector * FLASH_SECTOR_SIZE)` . diskio.c

`disk_write(BYTE pdrv, const BYTE *buff, DWORD sector, UINT count)` : diskio.c

Implements the flash **Read-Modify-Write (RMW)** sequence: diskio.c

1. Computes the 16 KB erase block boundary and byte offset within the block. diskio.c
2. Copies the entire 16 KB hardware block from physical flash into the static RAM buffer `s_blockBuffer` . diskio.c
3. Overwrites the target 512-byte sector inside `s_blockBuffer` with the incoming data. diskio.c
4. Erases the 16 KB physical flash block via DriverLib `FlashErase(blockPhysAddr)` . diskio.c
5. Programs the modified 16 KB buffer back to flash via `FlashProgram((uint32_t *)s_blockBuffer, blockPhysAddr, FLASH_ERASE_BLOCK_SIZE)`

. diskio.c

`disk_ioctl(BYTE pdrv, BYTE cmd, void *buff)` : diskio.c

Handles standard device control queries:

`CTRL_SYNC` : Confirms outstanding write operations are flushed. diskio.c + 1

`GET_SECTOR_COUNT` : Returns `512` sectors. diskio.c + 1

`GET_SECTOR_SIZE` : Returns `512` bytes. diskio.c + 1

`GET_BLOCK_SIZE` : Returns `32` (number of 512B sectors per 16 KB erase unit). diskio.c + 1

4. `diskio.h`

Header defining the standard ChaN FatFs hardware driver contracts and TI extensions. diskio.h

Key Contents

Types:

`DSTATUS` : Byte flag field indicating media status (`STA_NOINIT` , `STA_NODISK` , `STA_PROTECT`). diskio.h

`DRESULT` : Result enum (`RES_OK` , `RES_ERROR` , `RES_WRPRT` , `RES_NOTRDY` , `RES_PARERR`). diskio.h

Function Prototypes: Declarations for `disk_initialize` , `disk_status` , `disk_read` , `disk_write` , and `disk_ioctl` . diskio.h

TI Extensions: Defines `struct diskio_fxns` , `disk_register()` , and `disk_unregister()` for dynamic multicarrier driver binding. diskio.h

5. `ffcio.c` & `ffcio.h`

The C Run-Time Input/Output (CIO) translation bridge. It enables standard POSIX/C library file operations (`<stdio.h>`) to route through FatFs. ffcio.c + 3

Static State

`filTable[_NSTREAM]` : Static array of pointers to `FIL` structures indexed by low-level device file descriptors. ffcio.c

Functions

`ffcio_open(const char *path, unsigned flags, int llv_fd)` : ffcio.c

Allocates a dynamic `FIL` structure via `malloc(sizeof(FIL))` . ffcio.c

Translates POSIX access flags (`O_RDONLY` , `O_WRONLY` , `O_RDWR` , `O_CREAT` , `O_TRUNC` , `O_APPEND`) into FatFs open modes (`FA_READ` , `FA_WRITE` , `FA_OPEN_ALWAYS` , `FA_CREATE_ALWAYS` , `FA_OPEN_APPEND`). ffcio.c

Invokes `f_open()` and returns the mapped `dev_fd` descriptor. ffcio.c

`ffcio_close(int dev_fd)` : ffcio.c

Flushes and closes the file via `f_close()` , deallocates the `FIL` object, and clears `filTable[dev_fd]` . ffcio.c

`ffcio_read(int dev_fd, char *buf, unsigned count)` : ffcio.c

Calls `f_read()` and returns the actual number of bytes transferred. ffcio.c

`ffcio_write(int dev_fd, const char *buf, unsigned count)` : ffcio.c

Calls `f_write()` and returns the bytes written. ffcio.c

`ffcio_lseek(int dev_fd, fpos_t offset, int origin)` : ffcio.c

Adjusts the file position using `f_tell()` (for `SEEK_CUR`) or `f_size()` (for `SEEK_END`), then applies the new offset with `f_lseek()` . ffcio.c

`ffcio_unlink(const char *path)` : ffcio.c

Maps file deletion to `f_unlink()` . ffcio.c

`ffcio_rename(const char *old_name, const char *new_name)` : ffcio.c

Renames a target file using `f_rename()` . ffcio.c

`mkdir(const char *path, unsigned int mode)` and `rmdir(const char *path)` : ffcio.c

POSIX filesystem bindings for `f_mkdir()` and `f_unlink()` . ffcio.c

6. `ff.c` & `ff.h`

The core engine of the generic ChaN FatFs module (version R0.13a). It handles the FAT12/FAT16/FAT32/exFAT disk file structures, Master Boot Records (MBR), BIOS Parameter Blocks (BPB), directory trees, and cluster chain traversal. ff.c + 2

Core Internal Functions (Private)

`find_volume()` : Parses logical drive tokens (`"0:"`), initializes disk drivers, verifies the boot sector/BPB, and mounts the active partition table. ff.c

`move_window()` & `sync_window()` : Reads and flushes the intermediate single-sector memory cache window (`fs->win`) between FatFs and disk drivers. ff.c

`get_fat()` & `put_fat()` : Navigates and updates the File Allocation Table cluster link chains for files. ff.c

`dir_sdi()` , `dir_next()` , `dir_find()` : Directory parser routines navigating directory entry clusters to search, index, or append files. ff.c

`create_chain()` & `remove_chain()` : Allocates free clusters or truncates/deallocates existing cluster allocations. ff.c

Exported API Functions

`f_mount()` : Registers/unregisters volume work structures. ff.h

`f_open()` & `f_close()` : Opens/creates and finalizes file sessions. ff.h

`f_read()` & `f_write()` : Core block read/write operations with multi-sector direct streaming optimizations. ff.c + 1

`f_lseek()` : Repositions the read/write byte pointer. ff.h

`f_mkfs()` : Formats physical media partitions with a FAT layout (writes BPB, FAT tables, and clean root directories). ff.c + 1

`f_sync()` : Commits dirty file buffers and updates size/timestamp meta-entries into flash. ff.c + 1

7. `ffconf.h`

The static configuration header dictating compile-time feature flags, RAM usage, and performance characteristics of `ff.c` : ffconf.h

Configuration Parameter	Value	Architectural Impact
`FF_FS_READONLY` ffconf.h	`0` ffconf.h	Enables read/write access (includes `f_write`, `f_mkfs`, `f_sync`). ffconf.h
`FF_USE_MKFS` ffconf.h	`1` ffconf.h	Compiles in `f_mkfs()`, enabling automatic disk formatting on blank flash. ffconf.h
`FF_MIN_SS` & `FF_MAX_SS` ffconf.h	`512` ffconf.h	Fixed sector size of 512 bytes, matching low-level logical emulation. ffconf.h
`FF_FS_REENTRANT` ffconf.h	`1` ffconf.h	Enables multithreaded thread-safe access. ffconf.h
`FF_SYNC_t` ffconf.h	`SemaphoreHandle_t` ffconf.h	Associates synchronization objects with FreeRTOS mutex handles. ffconf.h
`FF_FS_TIMEOUT` ffconf.h	`1000` ffconf.h	Mutex wait timeout set to 1000 OS ticks. ffconf.h
`FF_USE_LFN` ffconf.h	`0` ffconf.h	Disables Long File Names (SFN 8.3 naming convention used) to minimize RAM/Flash footprint. ffconf.h
`FF_CODE_PAGE` ffconf.h	`932` ffconf.h	Configures OEM Code Page 932 (Japanese Shift-JIS). ffconf.h

8. `ffunicode.c`

Provides code conversion tables between Unicode and OEM double-byte/single-byte character sets (DBCS/SBCS). ffunicode.c

`uni2oem932[]` : Lookup array converting Unicode wide characters to Shift-JIS (CP932) multi-byte sequences. ffunicode.c

`oem2uni932[]` : Reverse lookup table decoding Shift-JIS chara Unicode units. ffunicode.c what is mount

Compiles when `FF_USE_LFN` is enabled or non-default OEM code pages require wide lookup conversion. ffunicode.c

In computing, **mounting** is the process of making a storage device or filesystem (such as a hard drive, USB flash drive, SD card, or network share) accessible to the operating system and applications through the system's directory hierarchy. fatsd.c + 1

How It Works

The Physical Media: Before a drive is mounted, the operating system might recognize the physical hardware (raw sectors and bytes), but it does not treat it as an active directory or folder. diskio.c + 1

The Mount Point: The operating system associates the storage device with a specific location—either a directory path (common in Linux/Unix/POSIX systems, e.g., `/mnt/usb` or `/media/data`) or a drive letter/prefix (such as `D:` in Windows, or `"0:"` / `"fat:"` in embedded systems). fatsd.c + 2

Reading Filesystem Structures: During mounting, the OS reads the filesystem's metadata (e.g., the Master Boot Record (MBR), BIOS Parameter Block (BPB), or superblock) to determine the layout, sector size, cluster chains, and root directory. Once verified, user-level file operations (like opening, reading, and writing files) become possible. ff.c + 2

Mounting in Embedded Systems (FatFs)

In embedded systems using libraries like **FatFs**, mounting attaches a filesystem management structure (`FATFS`) to a logical drive number: fatsd.c + 1

Command:

C

```
f_mount(&g_fatFs, "0:", 1);
```

Immediate Mount (`opt = 1` **):** FatFs initializes the low-level disk driver, checks the boot sector and partition table on the physical medium, and registers the filesystem object so files can be opened. fatsd.c + 2

Delayed Mount (`opt = 0` **):** FatFs simply registers the structure and only performs the physical media checks upon the first file access. ff.c + 1

Unmounting: Passing `NULL` (e.g., `f_mount(NULL, "0:", 0)`) safely closes the filesystem link, flushes pending buffers, and releases any synchronization locks. fatsd.c + 1

give example how to create a file and delete file read and write

In your project, file operations can be done in two ways: using standard C runtime functions via the TI C-IO bridge (`fopen` , `fwrite` , `fread` , `remove`), or using the native FatFs API (`f_open` , `f_write` , `f_read` , `f_unlink`). fatsd.c + 3

Below are complete examples for both approaches.

Method 1: Standard C Library (`<stdio.h>`)

Because your project registers the `"fat"` device prefix via `add_device()` , you can target the flash drive using `"fat:0:<filename>"` . fatsd.c + 2

C

```c
#include <stdio.h>
#include <string.h>

void stdio_file_example(void)
{
    FILE *fp;
    char writeData[] = "Hello from Internal Flash FAT FS!\n";
```

```c
    char readBuffer[64];
    size_t bytesWritten, bytesRead;

    const char *filePath = "fat:0:test.txt";

    /* 1. Create and Write to a File */
    fp = fopen(filePath, "w");
    if (fp == NULL) {
        // Failed to create file
        return;
    }

    bytesWritten = fwrite(writeData, 1, strlen(writeData), fp);
    fflush(fp);
    fclose(fp);

    /* 2. Read from the File */
    fp = fopen(filePath, "r");
    if (fp == NULL) {
        // Failed to open file for reading
        return;
    }

    bytesRead = fread(readBuffer, 1, sizeof(readBuffer) - 1, fp);
    readBuffer[bytesRead] = '\0'; // Null-terminate string
    fclose(fp);

    /* 3. Delete the File */
    if (remove(filePath) != 0) {
        // File deletion failed
    }
}
```

Method 2: Native FatFs API (`<fat_driver/ff.h>`)

You can also bypass the C-IO layer and call the FatFs primitives directly using drive prefix `"0:<filename>"` . ff.c + 1

c

```c
#include <fat_driver/ff.h>
#include <string.h>

void fatfs_direct_example(void)
{
    FIL fil;
    FRESULT fr;
    UINT bytesWritten = 0;
    UINT bytesRead = 0;
    char writeData[] = "Native FatFs write operation.\n";
    char readBuffer[64];

    const TCHAR *filePath = "0:sample.txt";

    /* 1. Create and Write to a File */
    // FA_CREATE_ALWAYS creates a new file or overwrites if existing
    fr = f_open(&fil, filePath, FA_CREATE_ALWAYS | FA_WRITE);
    if (fr != FR_OK) {
        // Handle open error
        return;
    }

    fr = f_write(&fil, writeData, strlen(writeData), &bytesWritten);
    f_close(&fil);

    /* 2. Read from the File */
    fr = f_open(&fil, filePath, FA_READ);
    if (fr != FR_OK) {
        // Handle read open error
        return;
    }

    fr = f_read(&fil, readBuffer, sizeof(readBuffer) - 1, &bytesRead);
    if (fr == FR_OK) {
        readBuffer[bytesRead] = '\0';
    }
    f_close(&fil);

    /* 3. Delete the File */
    fr = f_unlink(filePath);
    if (fr != FR_OK) {
        // Handle delete error
    }
}
```

Crucial Details for This Project

Drive Prefix: When using Standard C functions, always prefix with `"fat:0:"` so the C runtime routes through `ffcio.c`. When calling FatFs functions directly, use `"0:"`. fatsd.c + 4

Committing Data: Flash operations require explicit flushes (`fflush()` or `f_sync()`) or closing (`fclose()` or `f_close()`) to commit changes from RAM buffers into physical on-chip flash memory. fatsd.c + 3

Read-Modify-Write Overhead: Because `disk_write()` erases and reprograms a 16 KB block for each 512-byte sector update, batch your writes when possible to reduce wear and latency. diskio.c

how to use this project explain in detail with example

How This Project Works and Executes

To use this project on an **MSP432E401Y LaunchPad**, you build it using Texas Instruments Code Composer Studio (CCS) or ARM GCC with the SimpleLink MSP432E4 SDK. When flashed to the board, it goes through a specific startup sequence:

1. **Hardware & Kernel Startup** (`main_freertos.c`):

   `main()` runs first, calling `Board_init()` to initialize MCU system clocks and power domains. main_freertos.c

   It creates a detached POSIX thread `mainThread` with a stack size of 1024 bytes and priority `1`. main_freertos.c

   FreeRTOS takes over CPU scheduling via `vTaskStartScheduler()`. main_freertos.c

2. **Filesystem Registration & Partition Setup** (`fatsd.c`):

   In `mainThread`, `add_device("fat", ...)` registers the custom C-IO driver so standard C `<stdio.h>` library functions recognize paths starting with `"fat:"`. fatsd.c + 1

A UART display interface is opened to send log prints to a PC serial terminal (115200 baud). fatsd.c

`f_mount(&g_fatFs, "0:", 1)` mounts the internal flash partition. fatsd.c

If the MCU's flash at `0x000C0000` has never been formatted, `f_mount()` returns `FR_NO_FILESYSTEM`. The program catches this and invokes `f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf))` to lay down the boot sector, FAT tables, and root directory automatically. fatsd.c + 4

3. **Application File Operations:**

   It opens/creates `"fat:0:input.txt"`, writes a demo text string to it, flushes, and closes it. fatsd.c

   It reads data back from `input.txt` and writes it to `"fat:0:output.txt"`. fatsd.c

   Finally, it re-opens `output.txt` and dumps its contents out via UART to show the filesystem is working. fatsd.c

Step-by-Step: How to Run the Default Project

Step 1: Hardware Connections

Connect your **MSP432E401Y LaunchPad** (e.g., MSP-EXP432E401Y) to your PC using the USB cable (connected to the XDS110 debug port).

Step 2: Open Terminal

Open a serial terminal (PuTTY, Tera Term, minicom, or CCS Terminal).

Select the **XDS110 Class Application/User UART** COM port.

Set settings: **115200 baud**, **8 data bits**, **no parity**, **1 stop bit**, **no flow control**.

Step 3: Build & Flash

In CCS, import the project.

Build and flash the project onto your microcontroller.

Hit Run/Resume.

Expected Serial Output:

Plaintext

```
--- MSP432E4 Internal Flash FAT File System ---
No filesystem found. Formatting 256KB flash area...
Format complete. Re-mounting...
Drive 0 mounted successfully.
Creating "fat:0:input.txt"...
File created.
Copied 169 bytes to "fat:0:output.txt".
**************************************************************************
MSP432E401Y Internal Flash FAT Filesystem (256 KB Region)
Data read and written to on-chip flash memory successfully.
**************************************************************************
Drive 0 unmounted successfully.
```

How to Adapt It for Your Own Application

Here is a practical, detailed example showing how you can replace or extend `mainThread()` to log sensor readings or save configuration files.

Custom Example: An Environmental Data Logger

Suppose you want to record periodic sensor readings (temperature and timestamp) to a CSV file (`datalog.csv`) on the internal flash, read back old records, and keep the file updated.

C

```c
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <time.h>

#include <FreeRTOS.h>
#include <task.h>
#include <fat_driver/ff.h>
#include <fat_driver/ffcio.h>
#include <ti/display/Display.h>
#include <ti/drivers/GPIO.h>
```

```c
#include "ti_drivers_config.h"

#define DRIVE_PREFIX "fat:0:"
#define LOG_FILE_PATH DRIVE_PREFIX "datalog.csv"

static Display_Handle display;
static FATFS g_fatFs;

/* Helper to log a sensor record */
static void log_sensor_reading(float temperature, float humidity)
{
    FILE *fp = fopen(LOG_FILE_PATH, "a"); // "a" opens for appending[cite: 7]
    if (!fp) {
        Display_printf(display, 0, 0, "Error opening log file for writing!\n");
        return;
    }

    // Write CSV row: SecondsSinceBoot, Temperature, Humidity
    fprintf(fp, "%lu,%.2f,%.2f\n", (unsigned long)time(NULL), temperature, humidi

    // CRITICAL: Ensure dirty data in RAM buffer commits to internal flash[cite:
    fflush(fp);
    fclose(fp);
}

/* Helper to read back the log file */
static void print_log_file(void)
{
    FILE *fp = fopen(LOG_FILE_PATH, "r"); //
    if (!fp) {
        Display_printf(display, 0, 0, "No existing log found.\n");
        return;
    }

    char lineBuffer[64];
    Display_printf(display, 0, 0, "\n--- Stored Log Entries ---\n");
    Display_printf(display, 0, 0, "Timestamp,Temp,Humidity\n");

    while (fgets(lineBuffer, sizeof(lineBuffer), fp) != NULL) {
        Display_printf(display, 0, 0, "%s", lineBuffer);
    }

    fclose(fp); //[cite: 1, 7]
}

/* Main Thread replacement */
```

```c
void *mainThread(void *arg0)
{
    FRESULT fresult;

    GPIO_init(); //[cite: 1]
    Display_init(); //[cite: 1]

    /* 1. Register TI C-IO bridge */
    add_device("fat", _MSA, ffcio_open, ffcio_close, ffcio_read,
               ffcio_write, ffcio_lseek, ffcio_unlink, ffcio_rename); //[cite: 1]

    display = Display_open(Display_Type_UART, NULL); //[cite: 1]
    if (!display) while (1); //[cite: 1]

    Display_printf(display, 0, 0, "\n=== Starting Logger Demo ===\n");

    /* 2. Mount and Auto-Format Drive */
    fresult = f_mount(&g_fatFs, "0:", 1); //[cite: 1]
    if (fresult == FR_NO_FILESYSTEM) { //[cite: 1]
        Display_printf(display, 0, 0, "Formatting flash region...\n");
        BYTE workBuf[FF_MAX_SS]; //[cite: 1]
        f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf)); //[cite: 1]
        f_mount(&g_fatFs, "0:", 1); //[cite: 1]
    }

    /* 3. Read previous logs from older boots */
    print_log_file();

    /* 4. Append simulated sensor samples */
    Display_printf(display, 0, 0, "\nLogging new sensor data...\n");
    log_sensor_reading(24.5f, 55.2f);
    vTaskDelay(pdMS_TO_TICKS(500));
    log_sensor_reading(24.8f, 54.9f);
    vTaskDelay(pdMS_TO_TICKS(500));
    log_sensor_reading(25.1f, 54.6f);

    /* 5. Print out updated content */
    print_log_file();

    /* 6. Unmount cleanly before shut-off */
    f_mount(NULL, "0:", 0); //[cite: 1]
    Display_printf(display, 0, 0, "Drive unmounted.\n");
```

```
    return NULL; //[cite: 1]
}
```

Important Things to Keep in Mind When Using This Project

1. **Path Syntax Differences:**

   If you use `<stdio.h>` functions (`fopen` , `remove`), **always** prefix your paths with `"fat:0:"` (e.g., `"fat:0:data.bin"`) so the TI runtime directs it to `ffcio.c` [cite: 1, 7].

   If you use native FatFs functions (`f_open` , `f_unlink`), **only** use `"0:"` (e.g., `"0:data.bin"`). fatsd.c + 2

2. **Sector Size vs. Erase Block Size:** diskio.c

   The filesystem acts like it has **512-byte sectors**, but the MSP432E4 hardware can only erase in **16 KB blocks.** diskio.c

   Every time you modify or append to a file, `disk_write()` copies 16 KB of flash into RAM (`s_blockBuffer`), modifies the targeted 512-byte chunk, erases the 16 KB flash block, and writes it all back. diskio.c

   Best Practice: Group your writes into bigger chunks and avoid constantly opening, appending small 1-byte entries, and closing files in tight loops to prevent premature flash wear and sluggish performance. diskio.c

3. **Storage Boundary:**

   The disk is capped at **256 KB** (512 sectors $\times$ 512 bytes) starting at memory address `0x000C0000` . Do not place application firmware code or constants in linker script regions overlapping `0x000C0000` – `0x00100000` , as `f_mkfs`